

Tema 4: Estructuras Condicionales y Toma de Decisiones

MOMENTO 1: PRÁCTICA

EL CEREBRO DEL SISTEMA: LA BIFURCACIÓN DE RUTAS

Piensa por un momento en las decisiones diarias que tomas antes de salir de casa hacia el CEA Herman Ortiz Camargo. Miras el reloj, evalúas el clima y revisas tus bolsillos. Si está lloviendo, tomas un paraguas; si no, sales directamente. Si tienes clases a las 9 de la mañana y ya son las 8:45, decides tomar un mototaxi; de lo contrario, si tienes tiempo de sobra, decides ir caminando. Cada una de estas acciones humanas es el resultado de evaluar una o varias **condiciones**.

En el mundo del desarrollo de software y la ingeniería de sistemas, las computadoras no poseen intuición, pero son maestras absolutas en evaluar condiciones matemáticas y lógicas a velocidades incomprensibles. Cuando un programa de computadora se ejecuta, normalmente lee las instrucciones línea por línea, de arriba hacia abajo (secuencialmente, como vimos en los temas anteriores de Entradas y Salidas). Sin embargo, un sistema que solo va en línea recta es inútil para resolver problemas reales.

El Escenario en el Laboratorio: El Sistema de Control Biométrico

Imagina que, como proyecto de grado, implementamos un sistema de control de asistencia para los docentes utilizando un lector de huellas dactilares conectado a un microcontrolador ESP32 y una base de datos.

Cuando un profesor coloca su dedo sobre el escáner, el sistema captura la imagen y la convierte en un código numérico. En ese instante exacto, el procesador debe tomar una decisión crítica:

- **SI** el código leído es exactamente igual al código guardado en la base de datos... entonces, abre la cerradura electrónica y registra la asistencia en el sistema.
- **SI NO** (es decir, el código no coincide)... entonces, mantén la puerta cerrada, haz sonar una alarma de error y registra un intento fallido.

Esta capacidad de "romper" la línea recta y elegir caminos diferentes basados en preguntas lógicas es lo que dota a los programas informáticos de "inteligencia". A esto lo denominamos **Estructuras Condicionales** o Estructuras de Decisión.

MOMENTO 2: TEORÍA

ÁLGEBRA DE BOOLE Y OPERADORES LÓGICOS

Para enseñarle a una computadora a tomar decisiones, primero debemos aprender a hacerle las preguntas correctas. A diferencia del lenguaje humano, lleno de matices y dudas ("tal vez", "un poco"), el procesador (la CPU) solo entiende dos estados absolutos derivados del Álgebra de Boole (creada por el matemático George Boole): **VERDADERO (True / 1)** o **FALSO (False / 0)**. No existe término medio.

1. Los Operadores Relacionales (De Comparación)

Para generar un resultado Verdadero o Falso, necesitamos comparar datos. Al igual que usamos los operadores aritméticos (+, -, *, /) para hacer cálculos, utilizamos los operadores relacionales para establecer comparaciones directas entre variables.

Símbolo	Nombre Técnico	Ejemplo Práctico de Código	¿Qué pregunta le hace a la PC?
==	Igualdad Estricta	<code>contrasena == "1234"</code>	¿El valor de la izquierda es exactamente idéntico al de la derecha?
!=	Desigualdad (Diferente)	<code>usuario != "admin"</code>	¿El valor de la izquierda es distinto al de la derecha?
> / <	Mayor que / Menor que	<code>edad > 18</code>	¿El número es estrictamente mayor (o menor) que el otro?
>= / <=	Mayor o Igual / Menor o Igual	<code>horaIngreso <= 9</code>	¿El número es menor, o al menos igual, al límite establecido?

REGLA DE ORO DE LA ACADEMIA EDUCONECTRUBEN: El Error Fatal de = vs == El error número uno de los programadores principiantes es confundir el símbolo de Asignación (=) con el de Comparación (==).

Si escribes `if (estado = 1)`, no le estás preguntando a la máquina si el estado es 1. ¡Le estás **ORDENANDO** que el estado se convierta en 1! Esto destruye la lógica y provoca que la condición siempre se evalúe como verdadera. Para preguntar, **SIEMPRE** utiliza el doble igual:

```
if (estado == 1).
```

2. Los Operadores Lógicos (AND, OR, NOT)

En el mundo real, las decisiones rara vez dependen de una sola condición. Por ejemplo, para aprobar un módulo en el CEA, no solo necesitas tener una nota teórica alta, sino también haber entregado el proyecto práctico. Para combinar múltiples preguntas, utilizamos los Operadores Lógicos.

Operador	Símbolo en Código	Comportamiento (Tabla de Verdad)
AND (Y)	& &	Exige perfección. TODAS las condiciones deben ser Verdaderas para que el resultado final sea Verdadero. Si solo una falla, todo es Falso.
OR (O)		Es flexible. Con que AL MENOS UNA condición sea Verdadera, el resultado general es Verdadero. Solo es Falso si absolutamente todas fallan.
NOT (NO)	!	Es el rebelde. Invierte el resultado. Si algo era Verdadero, le pone un ! adelante y lo vuelve Falso.

3. Sintaxis de las Estructuras Condicionales

Una vez que sabemos hacer preguntas, construimos las bifurcaciones. En la mayoría de los lenguajes modernos (como C++, Java, JavaScript, PHP), se utiliza la palabra reservada `if` (si condicional en inglés) y `else` (sino / de lo contrario).

A. Condicional Simple (IF)

Solo ejecuta código si la condición se cumple. Si no se cumple, la computadora simplemente ignora el bloque y sigue su camino.

```
// Ejemplo: Otorgar bono de excelencia
Si (promedioAlumno >= 90) Entonces
    Imprimir("¡Felicidades! Tienes Beca Completa.")
    descuentoMensual = 100
Fin Si
```

B. Condicional Doble (IF - ELSE)

Establece dos caminos obligatorios. Si la condición es verdadera, entra por el primer camino. Si es falsa, entra obligatoriamente por el segundo.

```
// Ejemplo: Sistema de Votaciones CEA
Si (edadEstudiante >= 18) Entonces
    Imprimir("Usted está habilitado para emitir su voto en las urnas.")
    HabilitarPantallaVotacion()
Sino
    Imprimir("Error: Es menor de edad. Voto rechazado.")
    BloquearSistema()
Fin Si
```

C. Condicional Múltiple Anidada (IF - ELSE IF)

Sirve para evaluar cadenas de opciones sucesivas hasta encontrar una que sea verdadera. Es sumamente útil para categorizar datos.

```
// Ejemplo: Clasificación de notas técnicas
Si (notaFinal >= 90) Entonces
    Imprimir("Aprobado con EXCELENCIA")
Sino Si (notaFinal >= 61) Entonces
    Imprimir("Aprobado SATISFACTORIO")
Sino Si (notaFinal >= 51) Entonces
    Imprimir("Habilitado a SEGUNDA INSTANCIA")
Sino
    Imprimir("REPROBADO DEFINITIVO")
Fin Si
```

MOMENTO 3: VALORACIÓN

DISEÑO DE INTERFACES Y SEGURIDAD LÓGICA

En la ingeniería de software y el desarrollo de hardware, las estructuras condicionales no son un simple tema teórico; son la principal barrera de contención contra el caos.

Al desarrollar sistemas comerciales, la precisión de un `if` define el éxito o el fracaso de todo el proyecto institucional.

Pensemos en el proyecto del backend administrativo del Centro Educativo. Si un programador inexperto diseña una estructura condicional deficiente para procesar las inscripciones y coloca `if (cuposDisponibles > 0)` en lugar de validar estrictamente las bases de datos de paralelos, podría ocasionar una sobrepoblación en las aulas y el colapso del sistema de registros en el primer día de clases.

En el ámbito del hardware electrónico, los errores lógicos son literalmente físicos. Si estás programando el cerebro de una máquina industrial automatizada usando C++ para Arduino y configuras mal un operador OR (`||`) en vez de un AND (`&&`) en las condiciones de seguridad térmica de los motores, el sistema podría ignorar un sobrecalentamiento y **provocar un incendio o quemar los componentes permanentemente**. La rigurosidad matemática no es opcional, es nuestra máxima responsabilidad como técnicos.

El Fenómeno del Cortocircuito Lógico (Short-Circuit Evaluation)

Los procesadores modernos son increíblemente eficientes. Cuando evalúan una condición con el operador OR (`||`), saben que si la primera condición ya es Verdadera, el resultado total será Verdadero sin importar lo que siga. Por lo tanto, *la computadora simplemente no lee el resto del código*. Se "cortocircuita" y salta adentro del `if`. Esto ahorra valiosos ciclos de reloj y milisegundos de procesamiento en servidores de alta carga.

MOMENTO 4: PRODUCCIÓN

CUADERNO DE TRABAJO: LABORATORIO DE LÓGICA

Desarrolla las siguientes actividades prácticas para medir tu capacidad de abstracción algorítmica y depuración (debugging) de software.

Actividad 1: Traducción de Reglas de Negocio a Código

El director del CEA solicita que programemos un pequeño filtro en el sistema de inscripciones para el campeonato deportivo estudiantil. Las reglas son estrictas:

- Para que un estudiante se inscriba al campeonato, debe cumplir **ambas** condiciones: Su edad debe ser menor o igual a 20 años, y su estado de matrícula debe ser "Activo".

- Si no cumple con alguna de las dos, se le debe mostrar el mensaje "Inscripción Rechazada".

Escribe en tu cuaderno el código en pseudocódigo (utilizando *Si*, *Sino* y los operadores lógicos correctos) para resolver esta problemática.

Actividad 2: Prueba de Escritorio (Mental Tracking)

Eres el auditor de sistemas y debes encontrar qué resultado arrojará este bloque de código sin usar una computadora. Sigue las variables paso a paso. (Recuerda que la computadora evalúa los IF en orden, de arriba hacia abajo).

```
Entero saldoCuenta = 500
Entero montoRetiro = 600
Booleano cuentaBloqueada = Falso
Cadena mensajeFinal = ""

Si (cuentaBloqueada == Verdadero) Entonces
    mensajeFinal = "Comuníquese con el banco urgente."
Sino Si (montoRetiro > saldoCuenta) Entonces
    mensajeFinal = "Fondos insuficientes para la operación."
    saldoCuenta = saldoCuenta - 5 // Penalidad por intento fallido
Sino
    saldoCuenta = saldoCuenta - montoRetiro
    mensajeFinal = "Retiro exitoso. Tome su dinero."
Fin Si

Imprimir(mensajeFinal)
Imprimir("Saldo restante: " + saldoCuenta)
```

Responde en tu cuaderno:

- a) ¿Qué texto exacto se mostrará en pantalla gracias a `mensajeFinal`?
- b) ¿Cuál será el valor numérico final que quedará guardado en la variable `saldoCuenta`?

Actividad 3: Auditoría y Corrección de Errores (Debugging)

Un estudiante de otra institución intentó crear un sistema de acceso, pero el programa deja entrar a todo el mundo sin importar qué contraseña pongan. Analiza su código y descubre el "Error Fatal" (revisa la Regla de Oro del Momento 2).

```
Cadena contrasenaIngresada = "Admin99"

Si (contrasenaIngresada = "123456") Entonces
    Imprimir("ACCESO CONCEDIDO A LA BÓVEDA SECRETA")
Sino
    Imprimir("ALERTA DE INTRUSO")
Fin Si
```

Escribe en tu cuaderno por qué este código está mal estructurado y vuelve a escribirlo con la corrección exacta para que funcione como una verdadera validación de seguridad.

Actividad 4: Desafío de Diseño de Interfaces (Flowgorithm)

Abre el software *Flowgorithm* en la computadora del laboratorio. Diseña un diagrama de flujo interactivo que pida por pantalla la **Hora de Ingreso** (en formato 24 horas, ejemplo: 8, 9, 10).

Si la hora es **menor o igual a 9**, que imprima "Llegada Puntual. Pase por favor".

Si la hora es **mayor a 9**, que imprima "Atraso Registrado. Diríjase a la dirección".

(Asegúrate de que los cuadros de salida tengan mensajes amables y claros para el usuario final, respetando la experiencia de usuario UX que practicamos).